Serverless Computing
(Function as a Service)

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

NORTHEASTERN.

Learning Objectives

- Define serverless computing and distinguish it from VMs and containers as compute models.
- Explain the event-driven execution model and identify appropriate triggers.
- Describe how AWS Lambda, GCP Cloud Functions, and Azure Functions work, including their execution lifecycles.
- Evaluate the benefits and drawbacks of serverless in real-world cloud architectures.
- Apply best practices for writing, packaging, and deploying serverless functions.
- Choose between serverless, containers, and VMs for a given workload using a structured decision framework.
- Identify the broader serverless ecosystem beyond FaaS (API gateways, step functions, serverless databases).

What Is Serverless Computing?

What Is Serverless Computing?

- Serverless computing is a cloud execution model where the cloud provider dynamically manages the allocation and provisioning of servers. Despite the name, servers still exist — the key shift is that developers no longer need to think about them. You write a function, upload it, and the provider handles everything else: provisioning, scaling, patching, and availability.
- The term "serverless" is sometimes used broadly to describe any managed service where you do not manage infrastructure (for example, managed databases or managed message queues). In this lecture, we focus primarily on **Function as a Service (FaaS)**, the compute-specific form of serverless where you deploy individual functions that run in response to events.
- Serverless computing encourages a microservices-oriented design. Instead of deploying a monolithic application on a server, you decompose your application into small, independent functions. Each function does one thing, is triggered by one event type, and can be deployed and scaled independently.

The Evolution of Compute Abstraction

- Cloud computing has progressively abstracted away infrastructure details. Each step up the abstraction ladder lets developers focus more on business logic and less on operations.
- Each layer hides more of the underlying infrastructure. With serverless, the developer only provides application code. The cloud provider manages the runtime, operating system, scaling, patching, and availability.

```
EVOLUTION OF CLOUD COMPUTE

Physical Servers    Virtual Machines    Containers    Serverless

App                 App                 App           Function
Runtime             Runtime             Runtime
OS                  OS                  Libs
Hardware            Hypervisor          Container     You only
                                        Engine        write the
                                                      function

You manage:         You manage:         You manage:
Everything          OS + App            App + Libs    Just code

Unit of scale:      Unit of scale:      Unit of scale:  Unit:
Machine             Machine             Application     Function

Abstracts:          Abstracts:          Abstracts:      Abstracts:
Nothing             Hardware            OS              Runtime
```

What Makes Serverless Different from Other Managed Services?

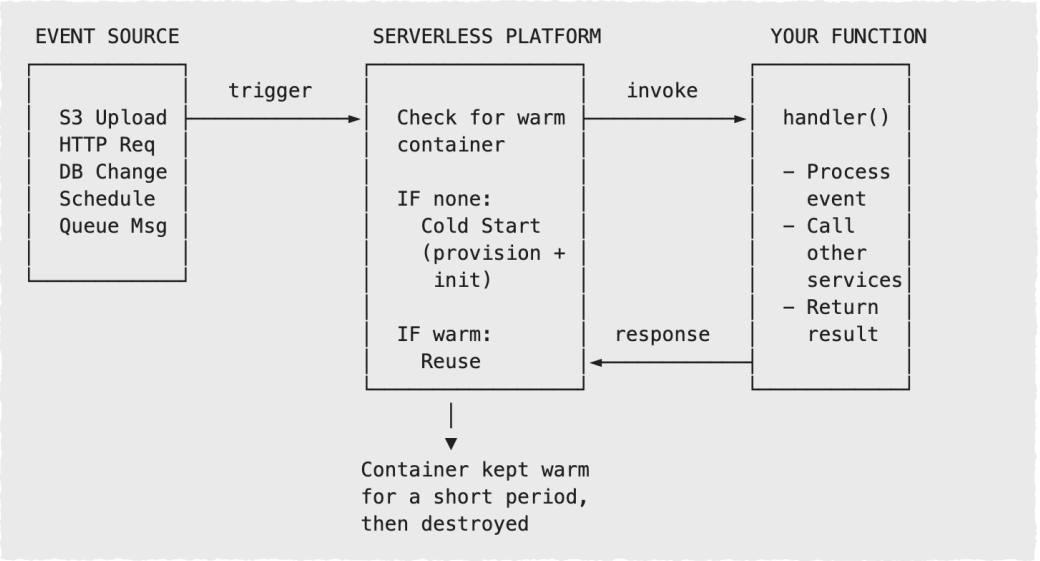
Serverless FaaS has a specific set of characteristics that distinguish it from simply using a managed service:

- **No server management.** You do not provision, configure, or maintain any servers or containers.
- **Event-driven execution.** Functions run only in response to triggers — they do not run continuously.
- **Automatic scaling.** The platform scales from zero to thousands of concurrent executions without configuration.
- **Pay-per-use billing.** You are charged only for the actual compute time your function consumes, measured in milliseconds. There is no charge for idle time.
- **Ephemeral execution.** Each function invocation is stateless. There is no guarantee that two invocations run on the same underlying infrastructure.

Event-Driven Architecture and Reactive Computing

Reactive Computing

- Serverless functions follow a reactive computing design. Code does not run continuously waiting for work — it is triggered by events or called directly via APIs.
- This is a fundamental shift from the traditional server model where an application process is always running.



Benefits of Serverless Computing

Benefits of Serverless Computing

- **No Server Management -** You do not provision, patch, or maintain any servers. The cloud provider handles the operating system, security patches, and runtime updates. This eliminates an entire category of operational work — there are no SSH sessions, no package upgrades, and no AMI rotations to worry about.
- **Automatic and Granular Scaling -** Serverless platforms scale your function automatically from zero instances to thousands of concurrent instances in response to demand. You do not configure auto-scaling groups or set scaling thresholds. Each incoming event gets its own isolated execution. Critically, serverless also scales *to zero* — if no events arrive, no resources are running and no costs accrue.
- **Pay-Per-Use Pricing -** You pay only for the compute time your function actually uses, typically measured in milliseconds. There is no charge for idle time. This is fundamentally different from VM-based pricing where you pay for the instance whether it is busy or idle. For workloads with sporadic or unpredictable traffic, this can mean dramatic cost savings.
- **Faster Time to Market -** Because infrastructure management is eliminated, developers can focus entirely on business logic. Deploying a new function can take minutes rather than the hours or days required to provision and configure servers. This accelerates iteration and experimentation.
- **Built-in High Availability and Fault Tolerance -** Serverless platforms run your functions across multiple availability zones by default. You do not need to configure load balancers, health checks, or failover mechanisms. The platform handles all of this transparently.
- **Reduced Operational Complexity -** There are no servers to monitor for disk space, memory leaks, or zombie processes. The operational surface area shrinks significantly. Monitoring shifts from infrastructure metrics (CPU, disk) to application metrics (error rates, latency, invocation counts).

Drawbacks and Challenges of Serverless Computing

Cold Start Latency

- When a function has not been invoked recently, the platform must provision a new execution environment — downloading your code, starting a runtime, and initializing your function. This process is called a **cold start** and adds latency to the first invocation.
- Cold start times vary significantly based on several factors:
  - **Runtime language:** Compiled languages like Go and Rust have faster cold starts. Java and .NET have slower cold starts due to JVM and CLR initialization. Python and Node.js fall in the middle.
  - **Package size:** Larger deployment packages take longer to download and unpack. A 50 MB Java JAR with Spring Boot will cold-start much slower than a 5 MB Python function.
  - **Memory allocation:** Higher memory allocations also provide more CPU, which can speed up initialization.
  - **VPC configuration:** Functions running inside a VPC historically had much longer cold starts due to ENI (Elastic Network Interface) attachment. AWS has improved this significantly, but VPC-attached functions still add some overhead.

```
Cold Start Breakdown (approximate):

┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐         │
│  │ Download │  │ Start    │  │ Init your │  │ Execute  │         │
│  │ code     │  │ runtime  │  │ code     │  │ handler  │         │
│  │          │  │ (JVM,    │  │ (global  │  │          │         │
│  │ ~100ms   │  │  Node,   │  │  scope,  │  │ Your code │         │
│  │          │  │  Python) │  │  imports) │  │ runs here │         │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘         │
│  ←── Cold start overhead ──→  ←── Billed duration ──→            │
│                                                                  │
│  Total cold start: 100ms (Go) to 5+ seconds (Java/Spring)        │
│  Warm invocation: typically < 10ms overhead                      │
└──────────────────────────────────────────────────────────────────┘
```

No Persistent In-Server State

- Each function invocation is stateless. You cannot store data in a global variable and expect it to be available on the next invocation — it might run on a different container. All state must be stored externally in databases, caches, or object storage.
- There is a nuance here: within a single container's lifetime, global variables *are* preserved between invocations. This is useful for connection pooling and caching, but you must never *depend* on it for correctness.

Vendor Lock-In

- Serverless functions are tightly integrated with the cloud provider's event sources, IAM model, deployment tools, and monitoring.
- Moving a Lambda function to GCP Cloud Functions is not a simple copy-paste — you need to rewrite event handling, change deployment configurations, and adapt to different service integrations.
- This coupling is deeper than with VMs or containers, which are more portable.

Execution Time Limits

- Serverless functions have maximum execution durations.
- AWS Lambda allows up to 15 minutes, GCP Cloud Functions up to 60 minutes (2nd gen), and Azure Functions has varying limits depending on the hosting plan.
- Long-running workloads like video transcoding, ML training, or batch ETL jobs may not fit within these constraints without being broken into smaller steps.

Debugging and Observability Challenges

- Debugging distributed, event-driven systems is harder than debugging a monolithic application on a single server.
  - You cannot SSH into a serverless function, attach a debugger, or inspect the filesystem.
  - Instead, you rely on structured logging (CloudWatch, Cloud Logging, Application Insights) and distributed tracing (X-Ray, Cloud Trace).
- Reproducing issues locally can be difficult because the execution environment differs from your laptop.

Loss of Server-Level Optimization

- With a traditional server, you can fine-tune the operating system, install custom system libraries, adjust kernel parameters, and optimize the runtime.
- With serverless, you operate within the provider's predefined execution environment.
- While custom container images offer some flexibility, the degree of control is still significantly less than managing your own servers.

AWS Lambda Deep Dive

Core Concepts

- **Function:** The unit of deployment. A function is a script or program that AWS Lambda runs. You upload your code (as a ZIP file or container image), and Lambda manages execution. Each function has a configuration that specifies the runtime, memory, timeout, IAM role, and environment variables.
- **Handler:** The entry point that Lambda calls when your function is invoked. You specify the handler when you create the function. Lambda passes two arguments to your handler: the **event** object (containing the trigger data) and the **context** object (containing metadata about the invocation, like the remaining execution time and the function's ARN).
- **Runtime:** The language environment that executes your function. Lambda natively supports Node.js, Python, Java, .NET, Go, Ruby, and custom runtimes via the Runtime API. You select the runtime when creating the function.
- **Event source:** The AWS service or external system that triggers your function. Event sources include S3, DynamoDB, SQS, SNS, API Gateway, EventBridge, Kinesis, and many others. Each event source provides a different event payload structure.
- **Layers:** Reusable packages of libraries, custom runtimes, or other dependencies that you can share across multiple functions. A function can use up to five layers. Layers reduce deployment package size and make it easier to manage shared dependencies.
- **Execution environment:** The isolated container that Lambda creates to run your function. This environment includes the runtime, your code, and any layers. Lambda may reuse an environment across multiple invocations (warm start) or create a new one (cold start).

Lambda Programming Model

```python
# Python Lambda handler example
import json
import boto3

# Global scope: executed once during cold start
# Reused across warm invocations — ideal for connections
s3_client = boto3.client('s3')

def handler(event, context):
    """
    event   - dict containing trigger data (S3 event, API Gateway request, etc.)
    context - object with runtime info:
              context.function_name
              context.memory_limit_in_mb
              context.get_remaining_time_in_millis()
              context.aws_request_id
    """
    # Process the event
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']

    # Use the globally-initialized client (connection reuse)
    response = s3_client.get_object(Bucket=bucket, Key=key)

    return {
        'statusCode': 200,
        'body': json.dumps({'message': 'Processed successfully'})
    }
```

Key principles of the programming model:

- **Handler** is the function Lambda calls. It receives the event and context.
- **Context** provides metadata: remaining time, function name, memory limit, request ID.
- **Logging** goes to CloudWatch Logs automatically. Use standard print/console.log statements.
- **Exceptions** — if the handler raises an exception, Lambda marks the invocation as failed and can retry depending on the event source configuration.
- **Concurrency** — each instance handles one request at a time. Lambda scales by adding more instances, not by adding threads to a single instance.
- **Statelessness** — treat each invocation as independent. Use external storage for state. However, you can (and should) use global scope for reusable resources like database connections.

Lambda Execution Lifecycle

```
┌──────────────────────────────────────────────────────────────┐
│                   LAMBDA EXECUTION LIFECYCLE                 │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  COLD START (first invocation or after idle timeout)         │
│                                                              │
│  ┌──────────┐   ┌──────────┐   ┌──────────────────────┐     │
│  │ Download │   │ Create   │   │ Run INIT code        │     │
│  │ code +   │──►│ execution│──►│ (imports, global vars,│    │
│  │ layers   │   │ env      │   │  DB connections)     │     │
│  └──────────┘   └──────────┘   └──────────────────────┘     │
│                                          │                   │
│                                          │                   │
│  ┌───────────────────────────────────────┘                   │
│  │                                                           │
│  ▼  INVOKE (every invocation)                                │
│                                                              │
│  ┌──────────────────┐                                        │
│  │ Run handler(event,│                                       │
│  │            context)│                                      │
│  └──────────────────┘                                        │
│            │                                                 │
│            ▼                                                 │
│                                                              │
│  ┌──────────────────┐                                        │
│  │ Return response  │                                        │
│  │ Container stays warm │──── Next invocation reuses        │
│  │ for ~5-15 minutes │      this container (warm)            │
│  └──────────────────┘                                        │
│            │ (no invocations)                                │
│            ▼                                                 │
│                                                              │
│  ┌──────────────────┐                                        │
│  │ SHUTDOWN         │                                        │
│  │ Container destroyed │                                     │
│  └──────────────────┘                                        │
└──────────────────────────────────────────────────────────────┘
```

Key Lambda Limits

Resource	Limit
Memory	128 MB to 10,240 MB (in 1 MB increments)
Timeout	Up to 15 minutes (900 seconds)
Deployment package (ZIP)	50 MB compressed, 250 MB uncompressed
Container image	10 GB
/tmp storage	512 MB to 10,240 MB
Concurrent executions	1,000 default (can request increase)
Burst concurrency	500–3,000 (varies by region)
Environment variables	4 KB total
Layers	5 per function
Invocation payload	6 MB (sync), 256 KB (async)

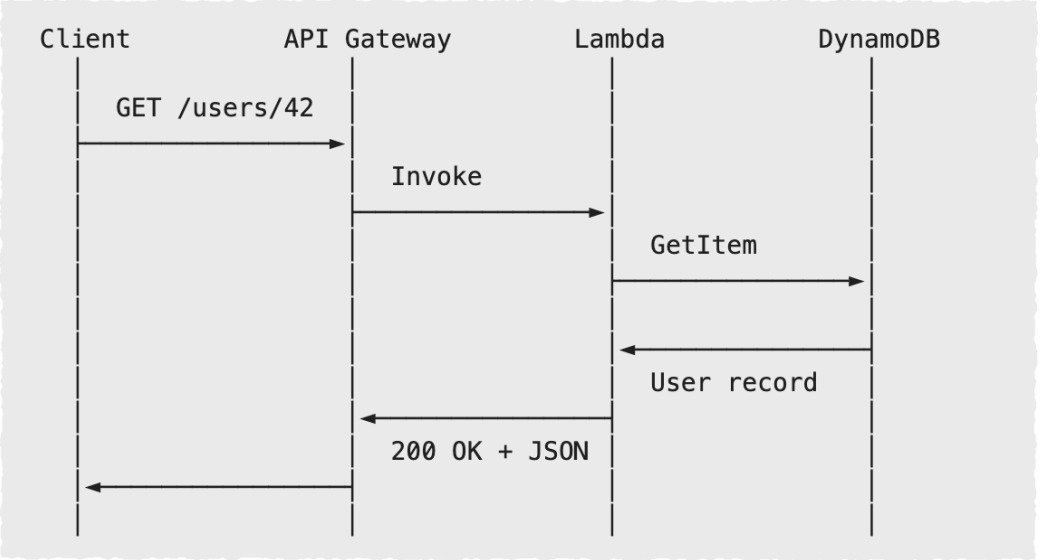
Lambda Pricing Model

- Lambda pricing has two components:
  - **Requests:** You are charged per invocation. The first 1 million requests per month are free. After that, it is $0.20 per million requests.
  - **Duration:** You are charged for compute time in 1-millisecond increments, based on the amount of memory allocated. For example, a function configured with 128 MB costs $0.0000000021 per millisecond.
- The key insight is that idle time costs nothing. A function that runs 1 million times per month for 200ms each at 128 MB costs roughly $0.63 in compute — dramatically cheaper than running an EC2 instance 24/7.

Serverless Architecture Patterns

REST API Backend

The most common serverless pattern. An API Gateway receives HTTP requests and routes them to Lambda functions, which query a database and return responses.



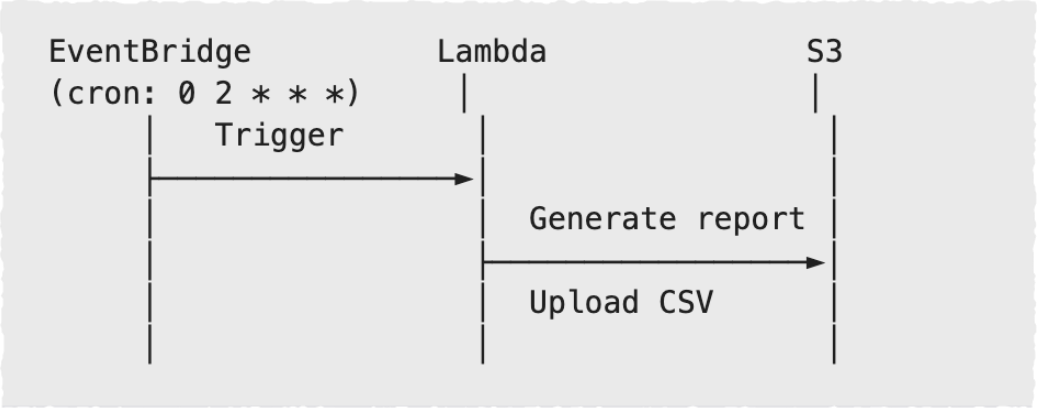
Event Processing Pipeline

An event triggers a chain of functions, each performing one step.

```
S3 Upload ──► Lambda: Validate ──► SQS ──► Lambda: Process ──► DynamoDB
                  │                                                  │
                  │ (invalid)                                        │
                  ▼                                                  ▼
              SNS: Alert Ops                                   SNS: Notify User
```

Scheduled Batch Jobs

A cron-like schedule triggers a function to perform periodic tasks.



When to Use (and When NOT to Use) Serverless

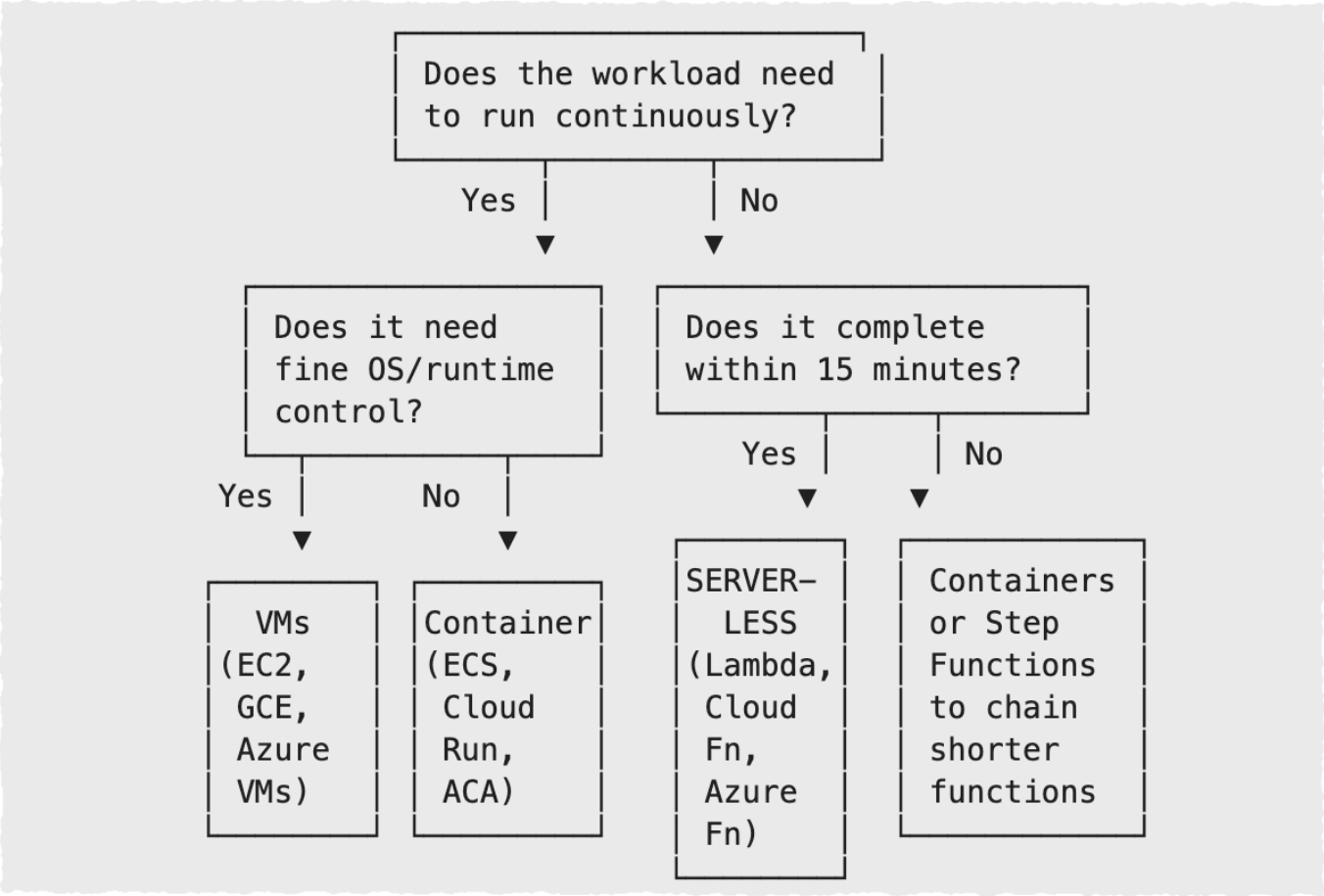
Good Fit for Serverless

- Event-driven workloads (file processing, webhooks, real-time notifications)
- REST APIs with variable or unpredictable traffic
- Scheduled tasks and cron jobs
- Lightweight data transformations and ETL
- Prototyping and MVPs (fast iteration, low cost at low scale)
- Workloads with spiky traffic patterns (occasional high bursts, long idle periods)

Poor Fit for Serverless

- Long-running computations (exceeding timeout limits)
- Workloads requiring persistent connections (WebSockets, unless using API Gateway WebSocket API)
- High-performance computing or GPU workloads
- Applications requiring fine-grained OS or runtime control
- Steady, high-throughput workloads (may be cheaper on reserved instances)
- Latency-sensitive applications that cannot tolerate cold starts

Decision Framework



Best Practices for Serverless Development

- **Optimize for Cold Starts**
  - Keep deployment packages small. Remove unused dependencies and dev-only libraries.
  - Prefer lightweight runtimes (Python, Node.js, Go) over heavy runtimes (Java with Spring) for latency-sensitive functions.
  - Use Lambda Layers to share common libraries across functions without bloating each deployment.
  - Use provisioned concurrency (AWS), minimum instances (GCP), or premium plan pre-warming (Azure) for latency-critical paths.
- **Leverage Container Reuse**
  - Initialize expensive resources (database connections, SDK clients, configuration) in the global scope, outside the handler. These persist across warm invocations.
  - Do not assume reuse — your code must be correct even when cold-starting, but it can be *faster* by reusing warm state.
- **Use Environment Variables for Configuration**
  - Never hard-code bucket names, table names, API endpoints, or secrets in your function code.
  - Use environment variables for operational parameters (bucket name, stage, region).
  - Use a secrets manager (AWS Secrets Manager, GCP Secret Manager, Azure Key Vault) for sensitive values like API keys and database credentials.
- **Write Stateless Functions**
  - Do not rely on the local filesystem (/tmp) for durable storage — it may be wiped between invocations.
  - Store all application state in external services (databases, caches, object storage).
  - Design each function as an independent, idempotent unit — if the same event is delivered twice (which happens with at-least-once delivery), the result should be the same.
- **Control Dependencies**
  - Pin your dependency versions. Lambda may update its built-in SDK, which can change behavior unexpectedly.
  - Bundle all dependencies with your deployment package for full control.
  - Use Lambda container images when you need a precise, reproducible environment.
- **Monitor and Observe**
  - Use structured logging (JSON format) so logs are machine-parseable.
  - Enable distributed tracing (AWS X-Ray, GCP Cloud Trace) to follow requests across functions and services.
  - Set CloudWatch alarms on error rates, throttling, and duration percentiles, not just invocation count.
  - Monitor concurrency usage to avoid hitting account-level throttling limits.

Serverless Security Considerations

- **Apply Least-Privilege IAM**
  - Each function should have its own IAM role with the minimum permissions required. A function that reads from S3 should not have permission to write to DynamoDB. This limits the blast radius if a function is compromised.
- **Secure the Event Sources**
  - Ensure that event sources like API Gateway endpoints have proper authentication and authorization. Use API keys, IAM authorization, Cognito (AWS), Identity Platform (GCP), or Azure AD to control who can invoke your functions.
- **Manage Secrets Properly**
  - Never store secrets in environment variables in plaintext. Use a secrets manager service and retrieve secrets at runtime. Many serverless frameworks support secrets manager integration natively.
- **Validate All Input**
  - Serverless functions receive input from external events. Never trust event data — validate and sanitize all input just as you would for any web application. Injection attacks can target serverless functions through event payloads.
- **Keep Dependencies Updated**
  - Vulnerabilities in third-party libraries are a common attack vector. Regularly scan your dependencies for known vulnerabilities and update them. Tools like Dependabot, Snyk, and npm audit can automate this.

Summary

Key Takeaways

- Serverless computing shifts operational responsibility from the developer to the cloud provider. You write functions, the provider handles everything else — provisioning, scaling, patching, and availability. The trade-off is reduced control in exchange for reduced operational burden and a pay-per-use cost model.
- The three major cloud providers each offer FaaS: AWS Lambda, GCP Cloud Functions, and Azure Functions. They share the same core model but differ in limits, pricing, ecosystem integration, and unique features (like Azure's declarative bindings or GCP's multi-concurrency per instance).
- Serverless is not the right choice for every workload. It excels at event-driven, sporadic, short-duration tasks and struggles with long-running, steady-state, or resource-intensive workloads. The decision framework — considering execution duration, traffic patterns, latency requirements, and portability needs — helps you choose the right compute model for each component of your system.

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS